

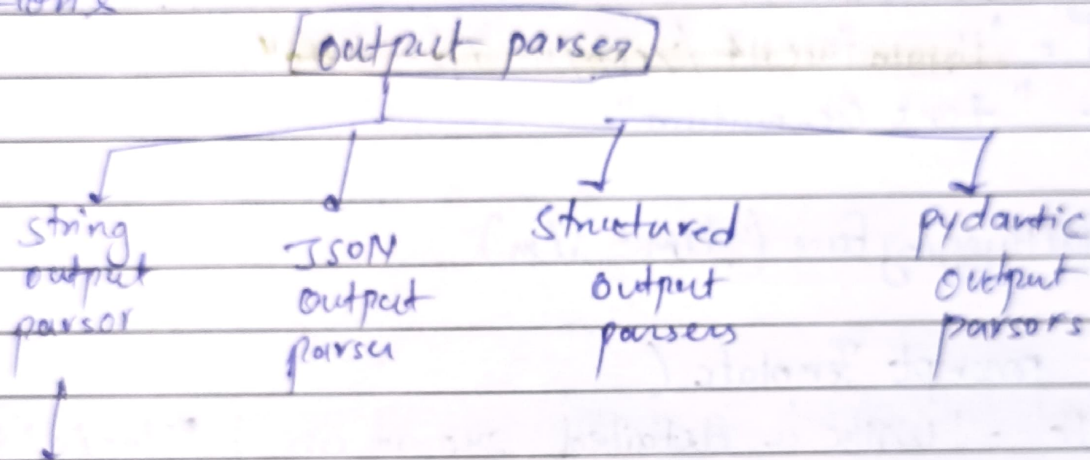
#

Date .....

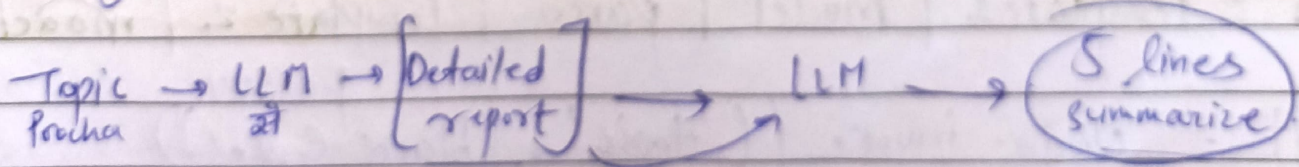
## Output parsers

→ cannot provide structured output

→ Output parsers in langchain help convert raw LLM responses into structured format like JSON, CSV, pydantic models and more. They ensure consistency, validation and ease of use in Applications.



simplest output parser in langchain  
it is used to parse the output of a language Model (LLM) and return it as a plain string.



## # Open source ! CODE

```

from langchain-huggingface import ChatHuggingface, HuggingfaceEndpoint
from dotenv import load_dotenv
from langchain-core.prompts import PromptTemplate
from langchain-core.output_parsers import StrOutputParser

```

```
load_dotenv()
```

```

llm = HuggingfaceEndpoint (
    repo_id = 'HuggingfaceH4/zephyr-7b-beta',
    task = "text-generation"
)

```

```
model = ChatHuggingface (llm=llm)
```

```

Template 1 = PromptTemplate (
    template = 'Write a detailed report on { "Topic" },
    input_variable = { "Topic" }
)

```

```

Template 2 = PromptTemplate (
    template = 'Write a 5 line summary on the following text. In { "text" }',
    input_variable = { "Text" }
)

```

```
Parser = StrOutputParser()
```

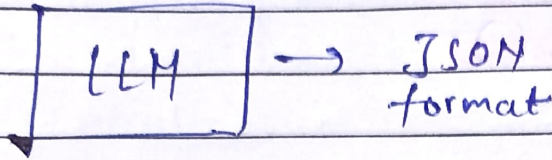
```
chain = Template 1 | model | parser | Template 2 | model | parser
```

```

result = chain.invoke ({ "topic": 'black hole' })
print (result)

```

② JSON Output Parser → JSON output parsers ek aisa mechanism hai jo LLM ka text output ko valid JSON format me convert aur validate karta hai.



→ JSON output parser ek aisa Mechanism hai jo LLM ke text output ko valid JSON format me convert aur validate karta hai.

```

ent: {
  "name" : "Shubham",
  "Skills" : ["python", "ML"]
}
  
```

Code: `from langchain-core output-parsers import JsonOutputParser.`

`Parser = JsonOutputParser()`

`template = PromptTemplate`

`template = 'Give me the name, age and city of a fictional person in {format instruction}'`

`input Variable = [],`

`partial_variable = {'format instruction': parser.get_format Instruction ()}`

`prompt = template * format ()`

`model`

`result = model.invoke (prompt)`

`print (result)`

↳ Instead of

`final_result = parser.parse (result.content)`

`print (final_result)`  
`print (type (final_result))`

## # Chain Method :-

Instead of using Piche wala code :-

```
chain = template | model | parser
result = chain.invoke()
print(result)
```

#

JSON output → it does n't force schema.  
parsers

③ Structured Output parser :-

→ schema provide karte hai  
jiske hisaab se hamara  
llm response karta hai !!

↳ is an output parser in langchain that helps extract  
structured JSON data from LLM responses  
based on predefined field schemas.

→ it works by defining a list of fields (Response  
schema) that the model should return,  
ensuring the output follows a structured format.

Code :-

from langchain.output\_parsers import Structured Output parser,  
Response Schema.

```
Schema = [
    Response Schema( name = 'fact 1', description = 'fact 1 about the topic' ),
    " " " 2 " " 2 "
    " " " 3 " " 3 "
]
```

same template = ' ' -

```
chain = template | model | parser
result = chain.invoke ({ 'topic' : 'black hole' })
print (result)
```

Disadvantage :-

no data validation ✓

for ex:

(str) name  
(int) age 35 yrs,  
(str) city

LLM output

string → 35 years -  
we have to accept it -

#### ④ pydantic Output Parser :-

↳ is a structured output parser in langchain that uses pydantic models to enforce schema validation when processing LLM responses.

#### Why Use pydantic Output Parser ??

- ✓ Strict Schema Enforcement → follow a well-defined structure.
- ✓ Type Safety → automatically convert LLM outputs into python objects.
- ✓ Easy Validation → uses pydantic's built-in validation to catch incorrect or missing data.
- ✓ Seamless Integration → works well with other langchain components.